

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
(ШКОЛА)

Департамент программной инженерии и искусственного интеллекта

ДОКУМЕНТАЦИЯ
по разработке прототипа игры с элементами обучения
в рамках учебной практики,
технологической (проектно-технологической)

Выполнил студент гр. Б9125-09.03.04пи

(подпись) Федотов Н. С.
(ФИО студента)

Отчет защищен с оценкой

Руководитель практики
доцент департамента ПИиИИ, к.ф.-м.н.
(должность, уч. степень, уч. звание)

(подпись) Ю.Е. Иванова
(И.О. Фамилия руководителя)
« 04 » июля 2026 г.

(подпись) Ю.Е. Иванова
(И.О. Фамилия руководителя)

Регистрационный №

Практика пройдена в срок
с « 19 » февраля 2026 г.
по « 04 » июля 2026 г.
на предприятии

« ____ » _____ 2026 г.

ДВФУ

(подпись) _____
(И.О. Фамилия)

Владивосток
2026

Оглавление

1 Введение.....	3
2 Основная часть	4
2.1 Описание проекта программного средства	4
2.2 Неформальная постановка задачи	4
2.3 Формальная постановка задачи	8
2.4 Алгоритм решения задачи.....	14
2.5 Спецификация данных.....	19
2.6 Спецификация функций	23
2.7 Проект программного средства	24
2.8 Описание данных программы.....	24
2.9 Алгоритм программы на языке PDL	30
2.10 Тесты.....	32
2.11 Тестирование программы.....	55
3 Заключение	55
4 Список использованных источников	63

1 Введение

Учебно-технологическая (проектно-технологическая) практика проходила в вузе на базе департамента программной инженерии и искусственного интеллекта Института математики и компьютерных технологий ДВФУ во втором семестре 2025-2026 учебного года с 19.02.2026 г. по 04.07.2026 г. (в объеме 3 з.е., 108 часов).

Целями учебно-технологической (проектно-технологической) практики являются: приобретение первичных практических умений и навыков по разработке проектов программных систем и проектной документации, а также знакомство с профессиональными задачами, решаемыми при создании программных систем.

Задачами учебно-технологической (проектно-технологической) практики являются:

- сбор и анализ требований заказчика к программному продукту;
- формализация предметной области программного проекта по результатам технического задания и экспресс-обследования;
- участие в проектировании компонентов программного продукта в объеме, достаточном для их конструирования в рамках поставленного задания;
- создание компонент программного обеспечения (кодирование, отладка, модульное и интеграционное тестирование);
- разработка тестового окружения, создание тестовых сценариев;
- разработка и оформление эскизной, технической и рабочей проектной документации.

2 Основная часть

2.1 Описание проекта программного средства

Проект программного средства представляет собой 2D-игру, реализованную на Python с использованием библиотеки Pygame. Программа предназначена для развития навыков быстрого набора текста и реакции пользователя в игровой форме, обеспечивая выбор сложности, автоматическую проверку ответов, подсчёт очков и сохранение рекорда, а также содержит интерактивную заметку с правилами игры для быстрого ознакомления новых пользователей.

Проект разработан с использованием принципов нисходящего проектирования; алгоритм программы основан на принципе объектно-ориентированного программирования.

Для реализации проекта были использованы Python, Pygame.

Работа выполнена командой из 3 человек Балушкина Евгения Николаевна, Федотов Никита Сергеевич, Лапошин Владислав Сергеевич. Моя роль в команде: председатель, генератор идей (по Белбину).

2.2 Неформальная постановка задачи

Разработать 2D-игру на Python с использованием библиотеки Pygame. В игре сверху вниз внутри области монитора падают сообщения со словами. Игрок вводит с клавиатуры полное слово в поле ввода и нажимает Enter для подтверждения, а кнопки интерфейса нажимает при помощи мыши. Если слово введено правильно - игрок получает 1 очко, слово исчезает с короткой анимацией. Если слово достигает нижней границы игровой области - игрок теряет 1 жизнь. Игра заканчивается, когда количество жизней становится равным 0. На начальном этапе у игрока 5 жизней.

Игра должна иметь:

–главное меню с выбором сложности (лёгкая, средняя, сложная) и

кнопкой старта;

– игровой экран с отображением счёта, жизней, падающих слов и поля ввода;

– экран окончания игры с итоговым счётом и возможностью начать заново или вернуться в меню.

Требования к особым случаям

1) нет активных слов. Игра продолжает работать, новые слова генерируются каждые 1.5 секунды;

2) несколько слов совпадают с ответом. Если в игровой области есть несколько слов, полная версия которых совпадает с введённым ответом, угаданным считается только одно слово. Удаляется слово, которое находится ближе всего к нижней границе (самое нижнее);

3) слово с пропуском. Если слово показано с пропущенной буквой (например, «с_ова»), а игрок вводит полное слово без пропуска («слова»), это считается правильным ответом. Проверка всегда идёт по полному слову, а не по отображаемому;

4) разный регистр. Игрок может вводить слово в любом регистре. «СЛОВО», «Слово», «слОвО» и «слово» считаются одинаковыми. Перед проверкой введённый текст и полное слово приводятся к нижнему регистру;

5) рекорд и текущий счёт. В правом верхнем углу игрового экрана отображается табличка, показывающая текущий счёт игрока (очки, набранные в текущей игре) и рекорд (лучший результат за все игры). Рекорд сохраняется между запусками игры. Если текущий счёт превышает сохранённый рекорд, рекорд обновляется. Рекорд также отображается на экране Game Over;

6) перезапуск игры. При нажатии «Заново» на экране Game Over счёт становится 0, жизни восстанавливаются до 5, все активные слова удаляются, генерация начинается заново. Выбранная сложность сохраняется;

7) возврат в меню. При нажатии «В меню» игра переходит в главное меню без сохранения текущего прогресса. Рекорд при этом сохраняется;

8) закрытие окна. При нажатии на крестик программа корректно завершается. Рекорд сохраняется в файл (например, *record.txt*) для использования при следующем запуске;

9) короткие слова. Для очень коротких слов (1-3 буквы) пропуск не ставится - такие слова всегда показываются полностью, чтобы игра оставалась справедливой;

10) анимация попадания. При правильном вводе слова слово не исчезает мгновенно. Запускается короткая визуальная анимация, длительностью 0.2-0.4 секунды, после чего удаляется из игровой области.

Требования к входным данным

Слова загружаются из текстового файла *words.txt*, который должен находиться в папке с игрой.

Формат файла *words.txt*:

- каждое слово записывается с новой строки;
- строки должны состоять только из русских букв (кириллица);
- ограничений по длине слова нет - используются любые слова из файла;
- пустые строки игнорируются.

Рекорд сохраняется в файл *record.txt* в папке с игрой. Файл создаётся автоматически при первом запуске и содержит одно число - максимальное количество очков.

Шрифт для отображения текста загружается из файла *.ttf* в папке *assets/fonts/*. Если файл шрифта отсутствует, используется системный шрифт, поддерживающий кириллицу. Если папка *assets/fonts/* отсутствует, программа пытается загрузить системный шрифт. Если системный шрифт не поддерживает кириллицу — выводится ошибка

Выбор сложности осуществляется пользователем в главном меню путём нажатия на соответствующую кнопку при помощи мыши. Доступны три уровня:

- 1) лёгкая — скорость падения слов 0.8 (условные единицы в секунду)
- 2) средняя — скорость падения слов 1.5
- 3) сложная — скорость падения слов 2.0

Требования к выводным данным

Игровое окно: 1280x720.

Главное меню

- а) название игры «Словопад»;
- б) кнопка «Старт» (начать игру);
- в) кнопка «Выход» (закрыть игру);
- г) три кнопки сложности: «Лёгкая», «Средняя», «Сложная» (выбранная выделяется);
- д) отображение текущего рекорда.

Игровой экран

- а) стол, монитор. Внутри монитора падают слова;
- б) на рамке монитора - пять лампочек жизнью;
- в) в правом верхнем углу - счёт и рекорд;
- г) под монитором - поле ввода с мигающим курсором;
- д) при правильном ответе слово исчезает с анимацией, счёт +1;
- е) при пропуске слова - гаснет лампочка, слово удаляется.

Экран Game Over

- а) текст «ИГРА ОКОНЧЕНА»;
- б) итоговый счёт и рекорд;
- в) кнопки: «Заново», «В меню».

Сообщения об ошибках

Критические ошибки (нет файла слов, некорректные данные) - сообщение в консоль, игра закрывается.

Недопустимые символы в поле ввода - временное сообщение «Только русские буквы».

Требования к формату данных

Файл words.txt: каждое слово с новой строки; только русские буквы; длина слова не ограничена; пустые строки игнорируются.

Файл record.txt: создаётся автоматически при первом запуске; содержит одно целое число - максимальное количество очков за одну игру; если файл повреждён или содержит некорректные данные, рекорд сбрасывается в 0.

Ввод игрока: допустимые символы: русские буквы (кириллица) в любом регистре; служебные клавиши: Backspace (удаление последнего символа), Enter (подтверждение ввода); максимальная длина ввода не ограничена (или ограничена длиной самого длинного слова в файле).

Регистр полностью игнорируется: перед проверкой введённый текст и полное слово приводятся к нижнему регистру. Игрок может вводить слово заглавными, строчными или смешанными буквами - любой вариант будет принят как правильный.

Отображение слова в облачке:

В 70% случаев слово показывается полностью.

В 30% случаев одна случайная буква в слове заменяется на символ подчёркивания `_`. Позиция замены выбирается случайным образом.

Для слов длиной 1-3 буквы пропуск не ставится — такие слова всегда показываются полностью.

2.3 Формальная постановка задачи

Входные данные:

Геометрия окон:

$WIDTH = 1280, HEIGHT = 720,$

где $WIDTH$ и $HEIGHT$ – ширина и высота окна соответственно.

Координаты монитора:

$X_m, Y_m, X_{1m}, Y_{1m} \in R$ - координаты в пределах которых находится экран изображения монитора.

Шрифт:

$\text{font} = \text{"CS_font.ttf"}$, где font – шрифт всех символов в игре.

Множество слов (файл words.txt):

$\text{word}_i = k_1 k_2 \dots k_n$, где $k_j \in \Sigma$, $j = 1, \dots, n$, где

$\Sigma = \{'a', 'б', 'в', \dots, 'я', 'А', 'Б', 'В', \dots, 'Я'\}$,

word_i слово считанное с файла words.txt

$W = \{(\text{text}_k, x_k, y_k) \mid \text{text}_k = \text{word}_i \neq "", k, i = 1, \dots, N, x_k \in [X_m, X_{1m}], y_k = 0\}$ –

множество слов с их координатами,

$\text{active_words} = \emptyset \subseteq W$ – множество активных слов (отображаемых на экране)

Сложность:

$\text{easy} = 0.8$,

$\text{normal} = 1.5$,

$\text{hard} = 2.0$,

где easy , normal , hard – константы обозначающие скорость падения слов на каждой сложности.

$D = \{\text{easy}, \text{normal}, \text{hard}\}$ – множество констант сложностей.

$\text{difficulty_name} \in D$ – текущая сложность.

Множество окон:

$S = \{\text{menu}, \text{game}, \text{game_over}\}$

Множество символов ввода:

$K_{\text{text}} = \{ \text{code}(k) \mid k \in \Sigma \}$,

где $\Sigma = \{'a', 'б', 'в', \dots, 'я', '0', '1', \dots, '9', '-'\}$,

$\text{code}(k)$ — код клавиши клавиатуры, соответствующий символу k .

$K_{\text{text}}^* = \{ w \mid w = k_1 k_2 \dots k_n, \text{ где } k_i \in K_{\text{text}}, n \in \mathbb{N}_0 \}$,

где w – одна из возможных комбинаций введенных клавиш.

$\text{user_text} \in K_{\text{text}}^*$ - ввод пользователя.

Управляющие клавиши:

$K_{\text{control}} = \{\text{Enter}, \text{Backspace}\}$ – коды используемых клавиш.

Мышь:

$M = (\text{Left Button}, \text{click}_m, x_m, y_m)$, где

Left Button – левая кнопка мыши,

click_m – состояние нажатия кнопки (1 если нажата, иначе 0),

x_m, y_m – координаты курсора мыши.

Состояния:

$\text{game_state} = \text{menu} \in S$,

где menu – состояние игры, при котором отображается меню

$\text{score} = 0 \in N_0$ – счёт игрока

$\text{lives} = 5 \in \{0, 1, 2, 3, 4, 5\}$ – количество жизней игрока

Рекорд:

$\text{record} \in N_0$ – рекорд игра, читается с resort.txt

Графические ресурсы:

$I = \{\text{back_room.jpg}, \text{monitor_r.jpg}, \text{monitor_menu.jpg}, \text{monitor_game.jpg}\}$

– множество изображений фона.

$B = \{\text{button_start.jpg}, \text{button_start_active.jpg}, \text{button_start_hover.jpg}, \text{button_exit.jpg}, \text{button_exit_active.jpg}, \text{button_exit_hover.jpg}, \text{button_restart.jpg}, \text{button_restart_active.jpg}, \text{button_restart_hover.jpg}, \text{button_menu.jpg}, \text{button_menu_hover.jpg}, \text{button_menu_active.jpg}, \text{button_easy.jpg}, \text{button_normal.jpg}, \text{button_hard.jpg}, \text{button_easy_active.jpg}, \text{button_normal_active.jpg}, \text{button_hard_active.jpg}, \text{button_easy_hover.jpg}, \text{button_normal_hover.jpg}, \text{button_hard_hover.jpg}\}$ – множество изображений кнопок

$Q = \{\text{game_title.jpg}, \text{choose_difficulty.jpg}, \text{note_off.png}\}$

Кнопки:

$\text{button_start} = (\text{background_b} = \text{button_start.jpg}, x1_b = x1_start, x2_b = x2_start, y1_b = y1_start, y2_b = y2_start)$ — кнопка начала игры

$\text{button_exit} = (\text{background_b} = \text{button_exit.jpg}, x1_b = x1_exit, x2_b = x2_exit, y1_b = y1_exit, y2_b = y2_exit)$ — кнопка выхода

button_easy = (background_b = button_easy.jpg, x1_b = x1_easy, x2_b = x2_easy, y1_b = y1_easy, y2_b = y2_easy) — кнопка выбора лёгкой сложности

button_normal = (background_b = button_normal.jpg, x1_b = x1_normal, x2_b = x2_normal, y1_b = y1_normal, y2_b = y2_normal) — кнопка выбора средней сложности

button_hard = (background_b = button_hard.jpg, x1_b = x1_hard, x2_b = x2_hard, y1_b = y1_hard, y2_b = y2_hard) — кнопка выбора высокой сложности

button_restart = (background_b = button_restart.jpg, x1_b = x1_restart, x2_b = x2_restart, y1_b = y1_restart, y2_b = y2_restart) — кнопка перезапуска игры

button_menu = (background_b = button_menu.jpg, x1_b = x1_menu, x2_b = x2_menu, y1_b = y1_menu, y2_b = y2_menu) — кнопка возвращения в меню

Интервал появления слов в миллисекундах:

$\Delta t = 1500$

Выходные данные:

game_state $\in \{\text{menu, game, game_over}\}$, где

game – состояние игры при котором отображается процесс игры.

game_over – состояние игры при котором отображается проигрыш.

score' $\in N_0$

record' $\in N_0$

lives' $\in \{0, 1, 2, 3, 4, 5\}$

active_words' $\subseteq W$

user_text' $\in K_{\text{text}}^*$, где

K_{text}^* — множество всех конечных последовательностей кодов символов из K_{text} ,

' – значение равное или отличное от начального.

M1 – фоновая музыка (“music.mp3”)

M2 – звук потери lives (“loose.mp3”)

M3 – звук угадывания слова (“guessing.mp3”)

M4 – звук окончания игры (“end.mp3”)

A1 – анимация исчезновения угаданного слова.

A2 — анимация потери жизни

images $\subseteq$ I – изображения отображаемые на экране.

buttons $\subseteq$ B – кнопки отображаемые на экране.

Связи

Переходы между окнами:

prepare(game_state) открывает окно game_state:

$(\text{game_state} = \text{menu}) \xrightarrow{\text{pressed}(\text{button_exit})} \text{завершение},$

$(\text{game_state} = \text{menu}) \xrightarrow{\text{pressed}(\text{button_start})} \text{prepare}(\text{game}),$

$(\text{game_state} = \text{game}) \xrightarrow{\text{lives}=0} \text{prepare}(\text{game_over}),$

$(\text{game_state} = \text{game_over}) \xrightarrow{\text{pressed}(\text{button_restart})} \text{prepare}(\text{game}),$

$(\text{game_state} = \text{game_over}) \xrightarrow{\text{pressed}(\text{button_menu})} \text{prepare}(\text{menu}),$

где $\text{pressed}(\text{button}) \Leftrightarrow (\text{click}_m = 1 \wedge (x_{1_b} \leq x_m \leq x_{2_b}) \wedge (y_{1_b} \leq y_m \leq y_{2_b}))$

Отрисовка элементов:

$\text{game_state} = \text{menu} \Rightarrow (\text{HEIGHT}, \text{WIDTH}, \text{background_menu},$
 $\text{buttons_menu})$

$\text{background_menu} = \{\text{back_room.jpg}, \text{monitor_r.jpg}, \text{monitor_menu.jpg}\} \subseteq$
I

$\text{buttons_menu} = \{\text{button_start}, \text{button_exit}, \text{button_easy}, \text{button_normal},$
 $\text{button_hard}, \text{record}\}$

$\text{game_state} = \text{game} \Rightarrow (\text{HEIGHT}, \text{WIDTH}, \text{background_game}, \text{score}, \text{lives},$
 $\text{active_words}, \text{user_text})$

$\text{background_game} = \{\text{back_room.jpg}, \text{monitor_r.jpg}, \text{monitor_game.jpg}\} \subseteq$
I

$\text{game_state} = \text{game_over} \Rightarrow (\text{HEIGHT}, \text{WIDTH}, \text{background_game_over},$
 $\text{buttons_game_over}, \text{score}, \text{record})$

$\text{background_game_over} = \{\text{back_room.jpg}, \text{monitor_r.jpg}, \text{monitor_game.jpg}\} \subseteq I$

$\text{buttons_game_over} = \{\text{button_restart}, \text{button_menu}\}$

Изменение вида кнопок:

$\forall \text{ button: } \text{hover}(\text{button}) \Rightarrow (\text{background_b}=\text{button_hover.jpg})$, где hover – наведение мыши на кнопку.

$\forall \text{ button: } \text{pressed_b}(\text{button}) \Rightarrow (\text{background_b}=\text{button_pressed.jpg})$, где pressed_b – кнопка нажата.

Генерация слов:

$\forall k \in \mathbb{N}: t_k = k \cdot \Delta t \exists \text{ word}_k \in W, \text{active_words}' = \text{active_words} \cup \{\text{word}_k\}$ – добавление слова в active_words раз в 1500мс.

Формирование отображения:

$\text{length}(\text{text}_k) \leq 3 \rightarrow \text{text}_k = \text{text}_k$,

$\text{length}(\text{text}_k) > 3 \rightarrow \text{text}_k = \text{text}_k$, с вероятностью 0.3,

$\text{text}_k = \text{replace_word}_k$, с вероятностью 0.7, где

text_k - текст слова word_k

$\text{length}(\text{text}_k) = m \Leftrightarrow \text{text}_k = k_1 k_2 \dots k_m$, где $k_j \in K_{\text{text}}, j = 1, \dots, m$.

$\text{replace_word} = w'_1 w'_2 \dots w'_m$:

$m = \text{length}(\text{text}_k)$,

$\exists i \in \{1, \dots, m\}: w'_i = _$,

$\forall j \in \{1, \dots, m\}, j \neq i: w'_j = \text{text}_{kj}$,

Движение слова:

$y_k' = y_k + \text{difficulty_name}$, где y_k – координата у слова word_k

Проверка слова:

$\text{lower}(\text{text}_k) = \text{lower}(\text{text}_k)$ и $\text{Enter} \in K_{\text{control}} \rightarrow (\text{score}' = \text{score} + 1, A1, M3, \text{active_words} \setminus \{\text{word}_k\})$, где

$\forall w = w_1 w_2 \dots w_n \in K^* \text{text}$:

$\text{lower}(w) = w'_1 w'_2 \dots w'_n$, где $w'_i \in \{'a', 'б', \dots, 'я'\} \Leftrightarrow w_i \in \{'A', 'Б', \dots, 'Я'\}$, $i = 1, \dots, n$.

Падение слова:

$y_k > \text{HEIGHT} \rightarrow (\text{lives}' = \text{lives} - 1, A2, M2, \text{active_words} \setminus \{\text{word}_k\})$

Сброс значений:

$\text{lives} = 0 \rightarrow M4$

$\text{score} = 0, \text{lives} = 5, \text{active_words} = \emptyset$

$\text{user_text} = ""$

$\text{record} = \max(\text{record}, \text{score})$

обновление record.txt

Завершение:

$(\text{game_state} = \text{menu}) \xrightarrow{\text{pressed}(\text{button_exit})} \text{завершение игры.}$

закрытие окна $\rightarrow$ завершение игры.

2.4 Алгоритм решения задачи

1. Начало.
2. Загрузить множество изображений (I): фон, монитор, экран меню, экран игры.
3. Загрузить шрифт.
 - 3.1. Если файл шрифта найден, то:
 - 3.1.1. Использовать этот шрифт.
 - 3.2. Если файл шрифта не найден, то:
 - 3.2.1. Использовать системный шрифт с поддержкой кириллицы.
 - 3.3. Если системный шрифт не поддерживает кириллицу, то
 - 3.3.1. Вывести сообщение об ошибке.
 - 3.3.2. Перейти к п. 10.
4. Загрузить список слов из файла words.txt в массив W.
5. Если файл не найден или пуст, то:
 - 5.1. Вывести сообщение об ошибке,
 - 5.2. Перейти к п. 10.

6. Загрузить рекорд из файла record.txt в переменную record.
7. Присвоить начальные значения: game_state = menu (состояние игры); score = 0 (счёт игрока); lives = 5 (кол-во жизней игрока); difficulty_name = normal (сложность); user_text = "" (строка ввода текста); running = true (флаг работы игры).
8. Запустить таймер появления слов с интервалом 1500 миллисекунд.
9. Начать главный игровой цикл (повторять, пока флаг running=true):
 - 9.1. Проверить состояние фоновой музыки:
 - 9.1.1. Если M1 не воспроизводится, то:
 - 9.1.1.1. Включить M1.
 - 9.2. Обработать события.
 - 9.2.1. Если получен сигнал закрытия окна, то:
 - 9.2.1.1. running = false.
 - 9.2.1.2. Перейти к п. 10.
 - 9.2.2. Если сработал таймер, то:
 - 9.2.2.1. Если game_state = game, то:
 - 9.2.2.1.1. Выбрать случайное слово из массива W.
 - 9.2.2.1.2. Вычислить случайную горизонтальную координату в пределах экрана монитора.
 - 9.2.2.1.3. Установить вертикальную координату равную верхней границе монитора.
 - 9.2.2.1.4. Вычислить скорость падения слов.
 - 9.2.2.1.5. Добавить слово с список активных слов (active_words).
 - 9.2.2.2. Если game_state = menu, то:
 - 9.2.2.2.1. Если нажата кнопка старт (button_start), то:
 - 9.2.2.2.1.1. game_state = game.
 - 9.2.2.2.2. Если нажата кнопка выхода (button_exit), то:
 - 9.2.2.2.2.1. running = false.

9.2.2.2.2. Перейти к п. 10.

9.2.2.2.3. Если нажата кнопка выбора сложности (Лёгкий (button_easy), средний (button_normal), сложный (button_hard)), то:

9.2.2.2.3.1. Установить соответствующее значение:
сложность = 0.8, 1.5, 2.0, difficulty_name =
easy/normal/hard.

9.2.2.3. Если game_state = game, то:

9.2.2.3.1. Если введен символ из мн-ва Σ (кириллица), то:

9.2.2.3.1.1. Добавить введенный символ в строку ввода (user_text).

9.2.2.3.2. Если нажата клавиша Backspace, то:

9.2.2.3.2.1. Удалить последний символ из user_text.

9.2.2.3.3. Если нажата клавиша Enter, то:

9.2.2.3.3.1. Перейти к п. 9.2.

9.2.2.3.4. Если нажата клавиша Escape, то:

9.2.2.3.4.1. Значения становятся game_state = menu; user_text = ""; active_words = $\emptyset$.

9.2.2.4. Если game_state = game_over, то:

9.2.2.4.1. Если нажата кнопка заново (button_restart), то:

9.2.2.4.1.1. Значения переменных становятся score = 0;
lives = 5; user_text = ""; active_words = $\emptyset$;
game_state = game.

9.2.2.4.2. Если нажата кнопка меню (button_menu), то:

9.2.2.4.2.1. Значения переменных становятся score = 0;
lives = 5; user_text = ""; active_words = $\emptyset$;
game_state = menu.

9.3. Проверить введенное слово:

- 9.3.1. Привести введённое пользователем слово (`user_text`) к нижнему регистру и очистить от пробелов.
- 9.3.2. Если `user_text` $\neq$ "", то:
 - 9.3.2.1. Цикл перебрать все слова в списке `active_words`, для каждого слова выполнить проверку:
 - 9.3.2.1.1. Если найдено одно или несколько совпадающих слов, то:
 - 9.3.2.1.1.1. Выбрать из найденных слов то, у которого координата Y наибольшая (самое нижнее).
 - 9.3.2.1.1.2. Увеличить счёт на 1.
 - 9.3.2.1.1.3. Отметить слово как угаданное (слово больше не участвует в проверках и не двигается).
 - 9.3.2.1.1.4. Установить таймер анимации.
 - 9.3.2.1.1.5. Запустить анимацию исчезновения.
 - 9.3.2.1.1.6. Воспроизвести звук угадывания M3, перейти к 9.3.2.2.
 - 9.3.2.1.2. Иначе (если не найдено),
 - 9.3.2.1.2.1. Перейти к следующему слову в п. 9.3.2.1.
 - 9.3.2.2. Очистить строку ввода (`user_text`).
- 9.4. Если `game_state = game`, то:
 - 9.4.1. Цикл перебрать все слова в списке (`active_words`):
 - 9.4.1.1. Если слово не угадано, то:
 - 9.4.1.1.1. Слово перемещается вниз: прибавить к вертикальной координате слова значение скорости.
 - 9.4.1.1.2. Если слово вышло за нижнюю границу экрана, то:
 - 9.4.1.1.2.1. Уменьшить количество жизней (`lives`) на единицу.
 - 9.4.1.1.2.2. Воспроизвести звук потери жизни (M2).
 - 9.4.1.1.2.3. Запустить анимацию потери жизни.

- 9.4.1.1.2.4. Удалить это слово из списка активных слов.
- 9.4.1.1.2.5. Перейти к следующему слову в п. 9.4.1.
- 9.4.1.2. Если слово угадано, то:
 - 9.4.1.2.1. Уменьшить таймер анимации на единицу.
 - 9.4.1.2.2. Если таймер анимации ≤ 0 , то:
 - 9.4.1.2.2.1. Удалить слово из списка `active_words`, перейти к п. 9.4.1.
 - 9.4.1.2.3. Перейти к следующему слову п. 9.4.1.
- 9.4.2. Если `lives = 0`, то:
 - 9.4.2.1. Обновить рекорд (присвоить макс значение между текущим рекордом и актуальных счётом).
 - 9.4.2.2. Записать рекорд в `record.txt`.
 - 9.4.2.3. Воспроизвести звук окончания игры (M4).
 - 9.4.2.4. Перейти в состояние окончания игры (`game_state = game_over`).
- 9.5. Отрисовать кадр.
 - 9.5.1. Нарисовать фон комнаты (`back_room.jpg`) и нарисовать рамку монитора (`monitor_r.jpg`).
 - 9.5.2. Если `game_state = menu`, то
 - 9.5.2.1. Нарисовать экран меню (`monitor_menu.jpg`).
 - 9.5.2.2. Иначе: Нарисовать экран игры (`monitor_game.jpg`).
 - 9.5.3. Если `game_state = menu`, то
 - 9.5.3.1. Нарисовать заголовки (`game_title.jpg`, `choose_difficulty.jpg`).
 - 9.5.3.2. Нарисовать кнопки игры: старт, выход, лёгкий, средний, сложный (`button_start.jpg`, `button_exit.jpg`, `button_easy.jpg`, `button_normal.jpg`, `button_hard.jpg`).
 - 9.5.3.3. Нарисовать текущий рекорд.
 - 9.5.4. Если `game_state = game`, то

- 9.5.4.1. Нарисовать текущий счёт.
- 9.5.4.2. Нарисовать жизни.
- 9.5.4.3. Цикл перебрать слова в списке `active_words`:
 - 9.5.4.3.1. Если длина слова больше трёх, то
 - 9.5.4.3.1.1. С вероятностью 70% отобразить слово полностью, а с вероятностью 30% отобразить слово с одной буквой заменённой на подчёркивание.
 - 9.5.4.3.2. Нарисовать слово на экране, перейти к п. 9.5.4.3.
- 9.5.4.4. Нарисовать поле ввода.
- 9.5.5. Если `game_state = game_over`, то:
 - 9.5.5.1. Нарисовать текст «ИГРА ОКОНЧЕНА».
 - 9.5.5.2. Нарисовать финальный счёт (`score`) и рекорд (`record`).
 - 9.5.5.3. Нарисовать кнопки: заново (`button_restart.jpg`), в меню (`button_menu.jpg`).
- 9.5.6. Обновить экран.
- 9.6. Выдержать паузу для поддержания частоты 60 кадров в секунду, перейти к п. 9.
- 10. Конец.

2.5 Спецификация данных

Входные данные:

Загрузка шрифта из файла `CS_font.ttf`

Ввод всех данных из файла `words.txt`. Слов в файле 100. Порядок данных: каждое слово на новой строке:

`<word1>`

`<word2>`

...

`<wordi>`

Ввод данных из файла record.txt. Формат: одна строка с целым числом:

<record>

Файлы изображений из папки images:

back_room.jpg - фон комнаты

monitor_r.jpg - рамка монитора

monitor_menu.jpg - экран меню

monitor_game.jpg - экран игры

Изображения кнопок:

button_start.jpg; button_exit.jpg; button_restart.jpg; button_menu.jpg; button_easy.jpg; button_normal.jpg; button_hard.jpg

И их вариации (_hover.jpg; _active.jpg)

Ввод всех аудиофайлов:

music.mp3, loose.mp3, guessing.mp3, end.mp3

Ввод с клавиатуры. Формат: строка из русских букв.

<user_text>

Также обрабатываются клавиши: Enter (подтверждение), Backspace (удаление), Escape (выход в меню).

Ввод с мыши, где (x_m, y_m) – координаты курсора мыши, $click_m$ – состояние мыши:

<Left Button>

Выходные данные:

В зависимости от значения game_state игра отображает одно из трёх окон:

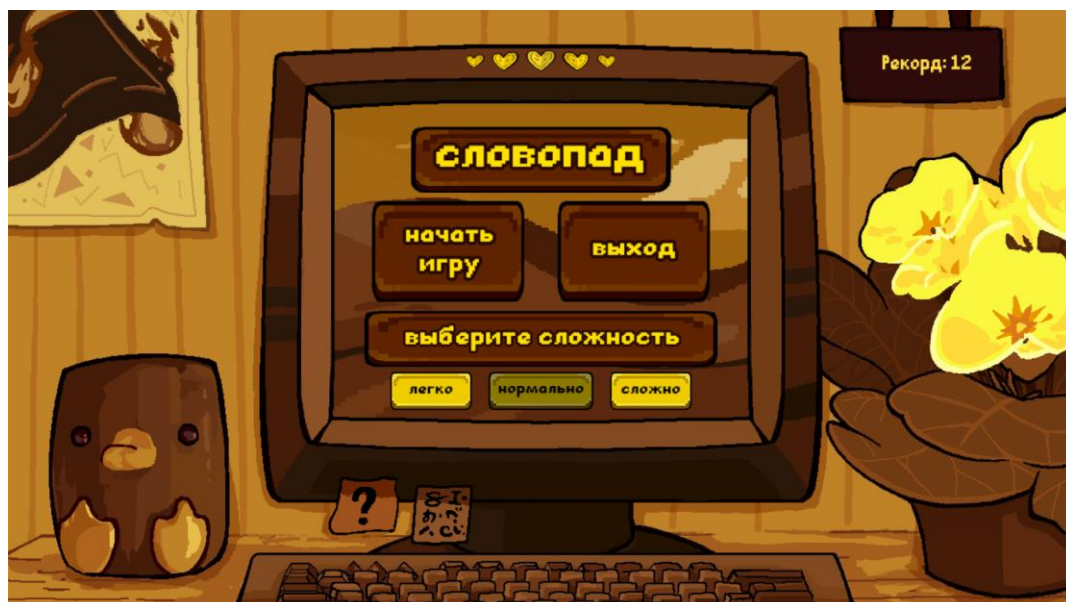


Рисунок 1 – главное меню (menu)



Рисунок 2 – игровой процесс (game)



Рисунок 3 – Экран окончания игры (game over)



Рисунок 4 – сообщение об ошибке, если вводятся не русские буквы

Вывод в файл record.txt - обновлённое значение рекорда. Формат: одна строка с целым числом.

<record>

Вывод звука: воспроизведение music.mp3, loose.mp3, guessing.mp3, end.mp3 в соответствующих событиях.

Сообщения, которые выводятся на консоль:

Сообщение 1: «Ошибка: системный шрифт не поддерживает кириллицу.»

Сообщение 2: «Ошибка: word.txt содержит некорректные данные в строке » + $\langle i \rangle$ + «. Допустимы только русские буквы и дефис.», где i – это номер строки, на которой находится слово, $1 \leq i \leq 100$.

Сообщение 3: «Ошибка: word.txt отсутствует или пуст.»

Сообщение 4: «Предупреждение: отсутствует изображение » + $\langle image_name \rangle$, где $image_name$ – это название файла изображения.

Сообщение 5: «Предупреждение: аудио недоступно.»

Сообщение 6: «Предупреждение: отсутствует аудиофайл » + $\langle M_num_name \rangle$ + «.», где M_num_name – это название аудиофайла.

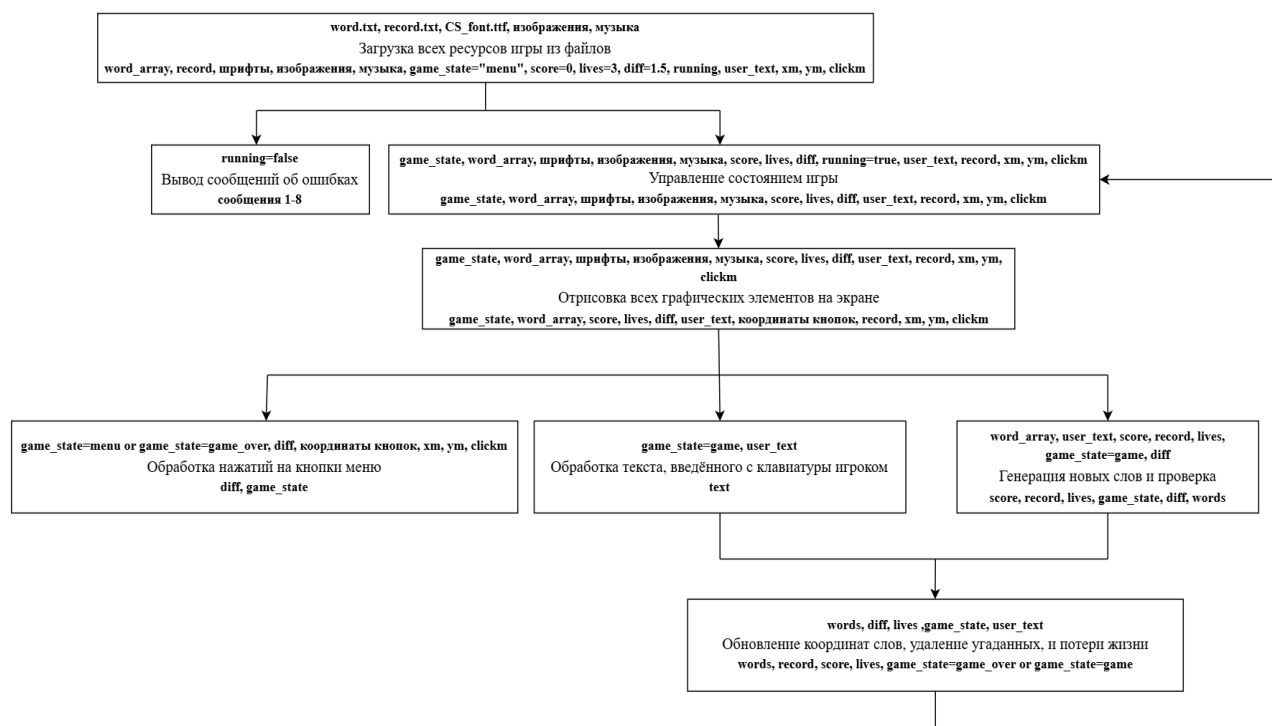
Сообщение 7: «Предупреждение: не удалось загрузить звук » + $\langle M_num_name \rangle$, где M_num_name – это название аудиофайла.

Сообщение 8: «Предупреждение: не удалось загрузить музыку » + $\langle M1_name \rangle$, где $M1_name$ – название аудиофайла M1.

2.6 Спецификация функций

1. Загрузка всех ресурсов игры из файлов
2. Вывод сообщений об ошибках
3. Обработка текста, введённого с клавиатуры игроком
4. Обработка нажатий на кнопки меню
5. Генерация новых слов и их проверка
6. Обновление координат слов, удаление угаданных, и потери жизни
7. Управление состоянием игры
8. Отрисовка всех графических элементов на экране

2.7 Проект программного средства



2.8 Описание данных программы

Таблица 1 – описание данных программы

Идентификатор	Назначение	Описание
FPS	Константа, обозначающая частоту кадров в секунду	integer
LETTERS	Строка, хранящая множество русских букв	string
MAX_LIVES	Константа, обознач. макс. кол-во жизней игрока	integer
save_record	Функция, сохраняющая переданное значение рекорда в файл record.txt	def save_record(v)
Локальный контекст функции save_record		

Продолжение таблицы 1

Идентификатор	Назначение	Описание
v	Входной параметр – значение рекорда, которое необходимо сохранить в файл (в п. 2.3, record)	integer
Word	Класс, хранящий в себе данные о каждом слове на экране	class
Переменные класса Word		
text	Поле класса, хранящее полное слово для проверки ответа	string
show	Поле класса, хранящее отображаемый текст (может содержать “_”), (в п. 2.3, replace_word)	string
f	Поле класса, хранящее шрифт для отрисовки слова (в п. 2.3, font)	pygame.font.Font
speed	Поле класса, хранящее скорость падения слова в пикселях за кадр (в п. 2.3, difficulty_name)	float
x	Поле класса, хранящее координату x левого верхнего угла слова	integer
y	Поле класса, хранящее координату y левого верхнего угла слова	integer
guessed	Логическая переменная-флаг, указывающая, угадано ли слово	boolean
anim_duration	Поле записи, хранящее длительность анимации исчезновения в кадрах	integer

Продолжение таблицы 1

Идентификатор	Назначение	Описание
anim_timer	Поле записи, хранящее текущее значение таймера анимации (обратный отсчёт)	integer
w	Свойство, возвращающее ширину слова с отступами	integer
h	Свойство, возвращающее высоту слова с отступами	integer
rect	Свойство, возвращающее прямоугольник слова (x, y, w, h)	pygame.Rect
__init__	Метод, инициализирующий новый экземпляр класса Word (в п. 2.3, active_words')	def __init__(self, text, f, area, speed):
Локальный контекст метода __init__		
self	Входной параметр – ссылка на текущий экземпляр класса Word	Word
text	Входной параметр – полное слово для проверки ответа (в п. 2.3, word _i)	string
f	Входной параметр – шрифт для отрисовки слова (в п. 2.3, font)	pygame.font.Font
area	Входной параметр – игровая область монитора (в п. 2.3, x _m , y _m , x _{1m} , y _{1m})	pygame.Rect
speed	Входной параметр – скорость падения слов (в п. 2.3, difficulty_name)	float

Продолжение таблицы 1

Идентификатор	Назначение	Описание
update	Метод, обновляющий координату у слова при движении вниз (в п. 2.3, y'_k)	def update(self):
Локальный контекст метода update (для класса Word)		
self	Входной параметр – ссылка на текущий экземпляр класса Word	Word
mark_guessed	Метод, отмечающий слово как угаданное и запускающий анимацию (в п. 2.3, A1 – анимация исчезновения)	def mark_guessed(self):
Локальный контекст метода mark_guessed		
self	Входной параметр – ссылка на текущий экземпляр класса Word	Word
update_animation	Метод, обновляющий таймер анимации и возвращающий признак завершения (в п. 2.3, A1 – анимация исчезновения, active_words{word _k })	def update_animation(self):
Локальный контекст метода update_animation		
self	Входной параметр – ссылка на текущий экземпляр класса Word	Word
draw	Метод, отрисовывающий слово с подложкой (в п. 2.3, отрисовка элементов)	def draw(self, screen):
Локальный контекст метода draw		

Продолжение таблицы 1

Идентификатор	Назначение	Описание
self	Входной параметр – ссылка на текущий экземпляр класса Word	Word
screen	Входной параметр – поверхность экрана, на которую выполняется отрисовка	pygame.Surface
r	Переменная, хранящая прямоугольник слова	pygame.Rect
alpha	Переменная, хранящая уровень прозрачности для анимации (в п. 2.3, A1)	integer
Surface	Переменная, хранящая временную поверхность для анимации	pygame.Surface
local_rect	Переменная, хранящая внутренний прямоугольник	pygame.Rect
text	Переменная, хранящая отрендеренное изображение текста	pygame.Surface
display_word	Функция, возвращающая отображаемое слово (полностью или с пропуском одной буквы), (п. 2.3 формирование отображения)	def display_word(w):
Локальный контекст функции display_word		
w	Входной параметр – исходное слово для преобразования	string
indices	Переменная, хранящая индексы букв в слове (позиции, которые можно заменить)	array of integer

Продолжение таблицы 2

Идентификатор	Назначение	Описание
I	Переменная, хранящая в себе случайный индекс выбранной для замены буквы	integer
submit	Метод, проверяющий введённое слово, начисляющий очки и обновляющий рекорд (в п. 2.3, проверка слова)	def submit(self):
Локальный контекст метода submit		
Self	Входной параметр – ссылка на текущий экземпляр класса Game	Game
typed	Переменная, хранящая нормализованный введённый текст (нижний регистр, без дефисов)	string
found	Переменная, хранящая список найденных слов, совпадающих с введённым ответом	array of Word
target	Переменная, хранящая самое нижнее слово среди найденных	Word
update	Метод, обновляющий игровую логику: движение и удаление слов, потерю жизней, завершение игры (в п. 2.3, проверка, движение и падение слова, сброс значений)	def update(self):
Локальный контекст метода update (для класса Game)		
self	Входной параметр – ссылка на текущий экземпляр класса Game	Game

Окончание таблицы 2

i	Переменная, хранящая в себе счётчик цикла для перебора сердечек (жизней)	integer
alive	Переменная, хранящая список слов, оставшихся после обновления (не удалённых)	Array of Word
w	Переменная, хранящая текущее слово при переборе массива words	Word

2.9 Фрагмент программного кода

“”” Сохранение рекорда “””

```
def save_record(v):
    open(path("record.txt"), "w", encoding="utf-8").write(str(v))
```

“”” Отрисовка слов, анимация угадывания и движение слов “””

```
class Word:
    def __init__(self, text, f, area, speed):
        self.text = text          # полное слово для проверки
        self.show = display_word(text) # отображаемый текст (с пропуском для
        # усложнения)
        self.f = f                # шрифт для отрисовки
        self.speed = speed         # скорость падения (от сложности)
        self.x = random.randint(area.left + 20, max(area.left + 20, area.right
        - self.w - 20)) # случайная позиция по горизонтали
        self.y = area.top - 40     # выше границы, чтобы не мигало при появлении
        self.guessed = False      # флаг: слово угадано (остановит движение)
        self.anim_duration = int(FPS * 0.3) # сколько кадров длится анимация
        # (0.3 сек)
        self.anim_timer = 0        # сколько кадров осталось до удаления

    @property
    def w(self):
        return self.f.size(self.show)[0] + 44 # ширина с отступами

    @property
    def h(self):
        return self.f.size(self.show)[1] + 24 # высота с отступами
```

```

@property
def rect(self):
    return pygame.Rect(self.x, self.y, self.w, self.h) # прямоугольник для
проверки столкновений

def update(self):
    if not self.guessed:
        self.y += self.speed # двигаем вниз, пока не угадано

def mark_guessed(self):
    self.guessed = True # останавливаем движение
    self.anim_timer = self.anim_duration # запускаем анимацию

def update_animation(self):
    if not self.guessed:
        return False
    self.anim_timer -= 1 # уменьшаем таймер каждый кадр
    return self.anim_timer <= 0 # True = пора удалять

def draw(self, screen):
    r = self.rect
    if self.guessed:
        alpha = max(0, min(255, int(255 * self.anim_timer / max(1,
self.anim_duration)))) # прозрачность падает
        surface = pygame.Surface((r.width + 8, r.height + 8),
pygame.SRCALPHA)
        local_rect = pygame.Rect(4, 4, r.width, r.height)
        pygame.draw.ellipse(surface, (*YELLOW, alpha), local_rect)
        pygame.draw.ellipse(surface, (*WHITE, alpha), local_rect, 2)
        text = self.f.render(self.show, True, BLACK)
        text.set_alpha(alpha)
        surface.blit(text, text.get_rect(center=local_rect.center))
        screen.blit(surface, (r.x - 4, r.y - 4))
        return

    pygame.draw.ellipse(screen, WHITE, r) # белый фон
    pygame.draw.ellipse(screen, GRAY, r, 2) # серая обводка
    text = self.f.render(self.show, True, BLACK)
    screen.blit(text, text.get_rect(center=r.center))

""" Замена символа в слове с вероятностью 70% при длине слова больше 3 """
def display_word(w):
    if len(w) <= 3 or random.random() > 0.3:
        return w
    indices = [i for i, ch in enumerate(w) if ch in LETTERS] # находим индексы
всех букв в слове
    if not indices:
        return w
    i = random.choice(indices) # выбор случайной буквы для замены
    return w[:i] + "_" + w[i + 1:]

""" Проверка ввода и начисление очков """

```

```

def submit(self):
    typed = normalize_word(self.text.strip())
    found = [w for w in self.words if not w.guessed and w.text.lower() ==
typed]
    if found:
        target = max(found, key=lambda w: w.y) # самое нижнее слово
        target.mark_guessed()
        self.score += 1 # +1 к счёту
        self.record = max(self.record, self.score) # обновление рекорда
        play_sound(self.snd_guess)
    self.text = ""

""" Двигает слова вниз, отнимает жизни за пропущенные, удаляет угаданные после
анимации и завершает игру при нуле жизней """
def update(self):
    if self.input_warning_timer > 0:
        self.input_warning_timer -= 1

    for i in range(MAX_LIVES): # обновляем анимацию всех сердечек
        if self.heart_anim_timers[i] > 0:
            self.heart_anim_timers[i] -= 1

    if self.game_state != "game": # если игра не актив, то ничего не обновляем
        return

    alive = [] # Список слов, которые ост в игре после этого кадра
    for w in self.words:
        if w.guessed:
            if not w.update_animation():
                alive.append(w)
            continue

        w.update()
        if w.rect.top > self.area.bottom: # вышло ли за границу области?
            if self.lives > 0:
                self.heart_anim_timers[self.lives - 1] = HEART_ANIM_DURATION
                self.lives -= 1
                play_sound(self.snd_lost)
            else:
                alive.append(w)

    self.words = alive

    if self.lives <= 0:
        self.lives = 0
        save_record(self.record)
        play_sound(self.snd_end)
        self.game_state = "game_over"

```

2.10 Тесты

Метод тестирования: белый ящик (покрытие всех ветвей).

Таблица 2 – Тестовые сценарии для покрытия ветвей

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
1	Вызов <code>path("assets", "img.png")</code> .	Возвращается путь относительно каталога файла программы.	<code>path()</code> : объединение каталога <code>__file__</code> с переданными частями пути.
2	<code>first_existing()</code> получает список, где первый путь существует.	Возвращается первый существующий путь.	<code>first_existing()</code> : ветвь <code>variant and os.path.exists(variant) == True</code> .
3	<code>first_existing()</code> получает только отсутствующие пути и <code>None</code> .	Возвращается <code>None</code> .	<code>first_existing()</code> : все варианты не подходят, завершение цикла без результата.
4	<code>asset_path("music.mp3")</code> при наличии файла в <code>sounds</code> или <code>audio</code> .	Возвращается найденный путь к ресурсу.	<code>asset_path()</code> : поиск ресурса в типовых папках проекта.
5	<code>asset_path("missing.png")</code> при отсутствии ресурса во всех папках.	Возвращается <code>None</code> .	<code>asset_path()</code> : ресурс не найден ни в одном варианте.
6	<code>font(26)</code> , файл <code>CS_font.ttf</code> существует в каталоге проекта.	Загружается пользовательский шрифт через <code>pygame.font.Font</code> .	<code>font()</code> : ветвь найденного файла шрифта.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
7	font(26), пользовательского шрифта нет, системный Arial поддерживает кириллицу.	Возвращается системный шрифт.	font(): ветвь fallback на pygame.font.SysFont и успешный render("Привет").
8	font(26), пользовательского шрифта нет, системный шрифт не рендерит кириллицу.	Печатается ошибка, pygame.quit(), sys.exit().	font(): исключение при проверке кириллицы.
9	normalize_word("Кошка—Мышка").	Возвращается «кошка-мышка».	normalize_word(): приведение к нижнему регистру и замена длинного тире на дефис.
10	Файл words.txt отсутствует. Запуск load_words().	В консоли: «Ошибка: words.txt отсутствует или пуст.»; завершение.	load_words(): обработка FileNotFoundError и выход из цикла кодировок.
11	words.txt пустой или содержит только пустые строки.	В консоли: «Ошибка: words.txt отсутствует или пуст.»; завершение.	load_words(): ветвь if not word: continue и отсутствие итогового списка words.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
12	words.txt в utf-8-sig: «СЛОВО», пустая строка, «кошка—мышка».	Возвращается список [«слово», «кошка-мышка»].	load_words(): успешная загрузка в utf-8-sig, игнорирование пустых строк, normalize_word().
13	words.txt содержит строку «слово1».	Печатается ошибка о некорректных данных в строке 1; завершение.	load_words(): any(ch not in WORD_CHARS for ch in word) == True.
14	words.txt сохранён в cp1251, при чтении utf-8-sig возникает UnicodeDecodeError.	После ошибки кодировки выполняется повторное чтение cp1251; слова загружены.	load_words(): except UnicodeDecodeError: continue.
15	record.txt отсутствует. Вызов load_record().	Возвращается 0.	load_record(): исключение при открытии файла.
16	record.txt содержит «abc». Вызов load_record().	Возвращается 0.	load_record(): исключение ValueError при int(...).
17	record.txt содержит «12». Вызов load_record().	Возвращается 12.	load_record(): успешное чтение целого числа.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
18	save_record(7), затем чтение record.txt.	Файл содержит строку «7».	save_record(): запись рекорда в файл.
19	display_word("дом") при любой вероятности пропуска.	Возвращается «дом», символ «_» не появляется.	display_word(): len(w) <= 3.
20	display_word("слово"), random.random() = 0.9.	Возвращается «слово».	display_word(): ветвь random.random() > 0.3, слово показывается полностью.
21	display_word("слово"), random.random() = 0.0, random.choice выбирает индекс 1.	Возвращается «с_ово».	display_word(): длинное слово, замена одной русской буквы на «_».
22	display_word("----"), random.random() = 0.0.	Возвращается «----».	display_word(): indices пустой, ветвь if not indices.
23	load_img("existing.png") при существующем корректном изображении.	Изображение загружается и convert_alpha() выполняется.	load_img(): успешная ветвь загрузки ресурса.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
24	<code>load_img("bad.png")</code> при существующем, но повреждённом изображении.	Исключение перехватывается, функция продолжает поиск; при отсутствии других вариантов возвращает <code>None</code> .	<code>load_img()</code> : <code>except Exception</code> внутри <code>pygame.image.load()</code> .
25	<code>load_img("missing.png")</code> вызывается впервые.	Печатается предупреждение об отсутствии изображения, возвращается <code>None</code> .	<code>load_img()</code> : ресурс не найден, <code>shown_name</code> ещё нет в <code>_missing_assets</code> .
26	<code>load_img("missing.png")</code> вызывается повторно.	Предупреждение повторно не печатается, возвращается <code>None</code> .	<code>load_img()</code> : <code>shown_name</code> уже находится в <code>_missing_assets</code> .
27	<code>button_images("button_start")</code> при наличии обычной, <code>hover</code> и <code>active</code> картинок.	Возвращается кортеж из трёх изображений.	<code>button_images()</code> : формирование вариантов имён и три вызова <code>load_img()</code> .
28	<code>init_audio()</code> , <code>pygame.mixer.init()</code> выполняется успешно.	<code>_audio_ready = True</code> .	<code>init_audio()</code> : успешная инициализация микшера.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
29	init_audio(), pygame.mixer.init() выбрасывает исключение.	Печатается предупреждение, _audio_ready = False.	init_audio(): ветвь except Exception.
30	load_sound("guessing.mp3"), _audio_ready = False.	Возвращается None без попытки загрузки файла.	load_sound(): ветвь if not _audio_ready.
31	load_sound("guessing.mp3"), аудио готово, файл не найден.	Печатается предупреждение об отсутствии аудиофайла, возвращается None.	load_sound(): file_path is None.
32	load_sound("guessing.mp3"), файл найден и Sound загружается.	Возвращается объект Sound.	load_sound(): успешная загрузка звука.
33	load_sound("guessing.mp3"), файл найден, но Sound выдаёт исключение.	Печатается предупреждение о невозможности загрузить звук, возвращается None.	load_sound(): except Exception при pygame.mixer.Sound.
34	play_sound(None).	Ничего не происходит, ошибки нет.	play_sound(): ветвь if sound == False.
35	play_sound(sound), sound.play() успешно.	Звук запускается.	play_sound(): ветвь if sound == True и успешный play().

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
36	play_sound(sound), sound.play() выбрасывает исключение.	Исключение подавляется, программа продолжает работу.	play_sound(): except Exception внутри play().
37	Button.draw(): курсор не наведён, active=False, изображений нет.	Рисуется серая кнопка и белый текст.	Button.draw(): img отсутствует, hovered=False, active=False, is_pressed=False.
38	Button.draw(): курсор наведён, hover_image отсутствует, изображений нет.	Рисуется синяя кнопка.	Button.draw(): hovered=True, fallback на рисование прямоугольника BLUE.
39	Button.draw(): active=True и active_image есть.	Отрисовывается active_image.	Button.draw(): show_active=True, img=active_image.
40	Button.draw(): кнопка pressed=True, курсор наведён, мышь зажата, active_image нет.	Рисуется активное состояние цветом YELLOW.	Button.draw(): is_pressed=True, но active_image отсутствует.
41	Button.draw(): image есть, hover и active не используются.	На экран выводится масштабированное image.	Button.draw(): img найден, ветвь screen.blit(transform. scale(...)).

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
42	Button.click(): MOUSEBUTTONDOWN левой кнопкой внутри rect.	pressed становится True, action пока не вызывается.	Button.click(): первая ветвь условия нажатия.
43	Button.click(): MOUSEBUTTONUP левой кнопкой внутри rect после pressed=True.	Вызывается action, pressed становится False.	Button.click(): отпускание внутри rect, вложенная ветвь action().
44	Button.click(): MOUSEBUTTONUP левой кнопкой вне rect после pressed=True.	action не вызывается, pressed становится False.	Button.click(): отпускание вне rect.
45	Button.click(): событие не является MOUSEBUTTONDOWN/MO USEBUTTONUP или кнопка не левая.	Состояние не меняется, action не вызывается.	Button.click(): событие игнорируется.
46	Word.update(): guessed=False, speed=2.	Координата y увеличивается на 2.	Word.update(): ветвь not self.guessed.
47	Word.update(): guessed=True.	Координата y не меняется.	Word.update(): ветвь guessed=True.
48	Word.mark_guessed().	guessed=True, anim_timer=anim_d uration.	Word.mark_guessed(): запуск анимации угаданного слова.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
49	Word.update_animation(): guessed=False.	Возвращается False.	Word.update_animation(): ранний выход для неугаданного слова.
50	Word.update_animation(): guessed=True, timer после уменьшения > 0.	Возвращается False, слово остаётся на экране.	Word.update_animation(): анимация ещё не завершена.
51	Word.update_animation(): guessed=True, timer после уменьшения <= 0.	Возвращается True, слово можно удалить.	Word.update_animation(): завершение анимации.
52	Word.draw(): guessed=False.	Рисуются обычное облачко со словом.	Word.draw(): ветвь обычной отрисовки слова.
53	Word.draw(): guessed=True, anim_timer > 0.	Рисуются исчезающее слово с alpha.	Word.draw(): ветвь анимации угаданного слова.
54	Game.load_music(), _audio_ready=False.	Возвращается False.	Game.load_music(): аудио не готово.
55	Game.load_music(), аудио готово, файл не найден.	Печатается предупреждение, возвращается False.	Game.load_music(): file_path is None.
56	Game.load_music(), файл найден, music.load успешно.	Возвращается True.	Game.load_music(): успешная загрузка музыки.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
57	Game.load_music(), music.load выбрасывает исключение.	Печатается предупреждение, возвращается False.	Game.load_music(): except Exception.
58	ensure_music_loop(): running=True, music_loaded=True, music.get_busy()==False.	Запускается pygame.mixer.music .play(-1).	ensure_music_loop(): все условия истинны.
59	ensure_music_loop(): running=False или music_loaded=False или музыка уже играет.	Музыка не запускается.	ensure_music_loop(): хотя бы одно условие ложно.
60	stop_music(): music_loaded=True, music.stop успешно.	Музыка останавливается без ошибки.	stop_music(): успешная остановка.
61	stop_music(): music_loaded=True, music.stop выбрасывает исключение.	Исключение подавляется.	stop_music(): except Exception.
62	stop_music(): music_loaded=False.	Вызов stop не выполняется.	stop_music(): ветвь без действия.
63	make_buttons(): изображения кнопок отсутствуют.	Создаются кнопки с default_size и центрируются.	make_buttons(): create_button использует default_size.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
64	make_buttons(): изображение кнопки найдено.	Размер кнопки берётся из изображения.	make_buttons(): create_button использует img.get_width()/get_height().
65	set_diff("Лёгкая").	diff_name="Лёгкая", diff=0.8.	set_diff(): выбор лёгкой сложности.
66	set_diff("Средняя").	diff_name="Средняя", diff=1.5.	set_diff(): выбор средней сложности.
67	set_diff("Сложная").	diff_name="Сложная", diff=2.0.	set_diff(): выбор сложной сложности.
68	start(): перед запуском score=9, lives=1, text непустой, words не пуст.	game_state="game", score=0, lives=5, timers=[0]*5, text="", words очищен.	start(): полный сброс новой игры.
69	menu(): игра запущена, text и words не пусты.	game_state="menu", text="", words очищен, record сохраняется.	menu(): возврат в меню.
70	quit(): running=True.	record сохраняется, музыка останавливается, running=False.	quit(): корректное завершение игры.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
71	spawn(): words_src содержит «слово», diff=2.0.	В words добавляется объект Word со скоростью 2.0.	spawn(): генерация нового падающего слова.
72	submit(): text=" СЛОВО ", в words есть два неугаданных слова «слово» с y=10 и y=50.	Помечается нижнее слово y=50, score+1, record обновляется, text очищается.	submit(): normalize_word(strip), found не пуст, max(found, key=y).
73	submit(): text="слово", совпадающее слово уже guessed=True.	score не меняется, text очищается.	submit(): фильтр not w.guessed исключает уже угаданное слово.
74	submit(): text="нет", совпадений нет.	score не меняется, text очищается.	submit(): ветвь found == [].
75	submit(): score=5, record=10, найдено слово.	score=6, record остаётся 10.	submit(): record=max(record, score), рекорд не уменьшается.
76	update(): input_warning_timer=3.	Таймер предупреждения уменьшается до 2.	update(): ветвь input_warning_timer > 0.
77	update(): input_warning_timer=0.	Таймер остаётся 0.	update(): ветвь input_warning_timer == 0.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
78	update(): heart_anim_timers=[2,0,0,0,0].	Первый таймер уменьшается до 1.	update(): уменьшение активного таймера сердца.
79	update(): game_state="menu".	После таймеров функция завершает работу, слова не обновляются.	update(): ранний return при game_state != "game".
80	update(): game_state="game", слово guessed=True, update_animation() возвращает False.	Слово остаётся в списке alive.	update(): ветвь угаданного слова с незавершённой анимацией.
81	update(): game_state="game", слово guessed=True, update_animation() возвращает True.	Слово удаляется из списка.	update(): ветвь завершённой анимации угаданного слова.
82	update(): обычное слово после update() остаётся внутри области.	Слово остаётся в active words.	update(): ветвь w.rect.top <= area.bottom.
83	update(): обычное слово вышло ниже области, lives=5.	Слово удаляется, lives=4, запускается таймер сердца slot 4, game_over не наступает.	update(): потеря жизни при lives > 1.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
84	update(): обычное слово вышло ниже области, lives=1.	lives становится 0, сохраняется record, играет end, game_state="game_over".	update(): потеря последней жизни и ветвь lives <= 0.
85	game_state="game", одновременно приходит событие SPAWN (таймер) и событие KEYDOWN (Enter).	Создается новое слово (spawn), вызывается submit() для проверки ввода. Ошибок нет.	event(): последовательная обработка списка событий pygame.event.get().
86	event(): событие pygame.QUIT.	Вызывается quit(), running=False.	event(): первая ветвь обработки закрытия окна.
87	event(): событие SPAWN при game_state="game".	Вызывается spawn().	event(): ветвь таймера появления слова.
88	event(): событие SPAWN при game_state="menu".	spawn() не вызывается, далее событие передаётся кнопкам меню.	event(): условие SPAWN and game_state == "game" ложно.
89	event(): game_state="menu", событие мыши по кнопке сложности.	Вызывается click() соответствующей кнопки.	event(): ветвь обработки меню.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
90	event(): game_state="game_over", событие мыши по кнопке «Заново» или «В меню».	Вызывается click() кнопки экрана окончания игры.	event(): ветвь обработки game_over.
91	event(): game_state="game", TEXTINPUT="Ab-1".	В text добавляется «а-»	event(): TEXTINPUT, есть допустимые символы и есть недопустимые.
92	event(): game_state="game", TEXTINPUT="абв".	В text добавляется «абв», предупреждение не включается.	event(): TEXTINPUT, allowed == normalized.
93	event(): game_state="game", TEXTINPUT="123".	text не меняется, input_warning_timer =FPS.	event(): TEXTINPUT, allowed пустой и allowed != normalized.
94	event(): game_state="game", KEYDOWN Backspace, text="кот".	text становится «ко».	event(): ветвь pygame.K_BACKSP ACE.
95	event(): game_state="game", KEYDOWN Enter.	Вызывается submit().	event(): ветвь K_RETURN / K_KP_ENTER.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
96	event(): game_state="game", KEYDOWN Escape.	Вызывается menu(), переход в меню.	event(): ветвь K_ESCAPE.
97	event(): game_state="game", KEYDOWN другой клавиши.	Состояние не меняется.	event(): KEYDOWN без подходящей клавиши.
98	draw_text("Текст", center=True).	Текст размещается по центру координат.	draw_text(): ветвь center=True.
99	draw_text("Текст", center=False).	Текст размещается от левого верхнего угла.	draw_text(): ветвь center=False.
100	draw_input(): input_warning_timer=0.	Рисуется поле ввода без предупреждения.	draw_input(): ветвь без сообщения «Только русские буквы».
101	draw_input(): input_warning_timer>0.	Рисуется поле ввода и предупреждение.	draw_input(): ветвь отображения предупреждения.
102	heart_surface(slot), все frames для выбранного варианта None.	Возвращается None.	heart_surface(): ветвь if not any(frames).
103	heart_surface(slot, force_state="off").	Выбирается последний кадр HEART_FRAMES- 1.	heart_surface(): принудительное выключенное состояние.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
104	heart_surface(slot, force_state="on").	Выбирается первый кадр.	heart_surface(): принудительное включённое состояние.
105	heart_surface(slot), lives > slot_index и timer=0.	Выбирается первый кадр живого сердца.	heart_surface(): is_alive=True.
106	heart_surface(slot), timer>0.	Выбирается промежуточный кадр анимации.	heart_surface(): ветвь анимируемого сердца.
107	heart_surface(slot), lives <= slot_index и timer=0.	Выбирается последний кадр погасшего сердца.	heart_surface(): ветвь выключенного сердца без force_state.
108	heart_surface(slot), выбранный кадр None.	Возвращается None.	heart_surface(): ветвь if img is None.
109	heart_surface(slot 0), scale=0.7, кадр найден.	Возвращается уменьшенная поверхность.	heart_surface(): ветвь scale != 1.0.
110	heart_surface(slot 2), scale=1.0, кадр найден.	Возвращается поверхность без масштабирования.	heart_surface(): ветвь scale == 1.0.
111	draw_decor_note(): note_off_img существует.	Изображение записки выводится на экран.	draw_decor_note(): ветвь if self.note_off_img.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
112	draw_decor_note(): note_off_img=None.	Ничего не рисуется.	draw_decor_note(): отсутствие изображения записки.
113	draw_hearts(): все heart_surface() возвращают None.	Функция завершается без отрисовки.	draw_hearts(): ветвь if not any(surfaces).
114	draw_hearts(): часть поверхностей None, часть существует, y_center задан.	Рисуются существующие сердца, пропуски учитываются по spacing.	draw_hearts(): ветви if s и else внутри цикла, baseline_y берётся из y_center.
115	draw_hearts(): поверхности существуют, y_center=None.	baseline_y вычисляется как 50 + ref_h // 2.	draw_hearts(): ветвь y_center is None.
116	draw(): bg есть.	Фон комнаты выводится на экран.	draw(): ветвь if self.bg.
117	draw(): bg=None.	Фон комнаты не выводится, ошибок нет.	draw(): отсутствие bg.
118	draw(): game_state="menu", menu_bg есть, wordfall_img есть, difficulty_img есть.	Рисуется экран меню с картинками заголовка и сложности.	draw(): ветви меню с изображениями.

Продолжение таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
119	draw(): game_state="menu", wordfall_img=None, difficulty_img=None.	Рисуются текст «СЛОВОПАД» и текст «Сложность».	draw(): fallback-ветви меню без изображений.
120	draw(): game_state="game", game_bg есть, в words есть слово.	Рисуются игровой экран, слова внутри clip области, счёт, рекорд, сердца и поле ввода.	draw(): ветвь игрового состояния.
121	draw(): game_state="game_over".	Рисуются экран окончания игры, рекорд, погасшие сердца и кнопки.	draw(): ветвь game_over.
122	draw(): monitor есть.	Монитор выводится поверх фона.	draw(): ветвь if self.monitor.
123	draw(): monitor=None.	Монитор не выводится, ошибок нет.	draw(): отсутствие monitor.
124	run(): running=True на одной итерации, затем quit() меняет running=False.	Выполняются ensure_music_loop, tick, обработка событий, update, draw; после выхода pygame.quit().	run(): цикл while self.running и завершение.

Окончание таблицы 2

№	Input / действия	Ожидаемый Output / состояние	Проверяемое условие / ветвь
125	В words есть два разных слова с одинаковым максимальным значением y (координата низа), оба совпадают с введенным текстом.	max(found, key=lambda w: w.y) возвращает первое из найденных (т.к. max стабилен при равных ключах). Помечается одно слово, score+1.	submit(): max(found, key=lambda w: w.y) при равенстве координат.
126	game_state="menu" или game_state="game_over", пользователь нажимает клавиши с буквами.	Текст в поле ввода (self.text) не изменяется, символы не добавляются.	event(): обработка TEXTINPUT происходит только в ветви elif self.game_state == "game"

Таблица 3 – Матрица покрытия ветвей

Условие / ветвь	Тесты
path(): построение пути относительно __file__	1
first_existing(): найден существующий путь / путь не найден	2, 3
asset_path(): ресурс найден / не найден	4, 5
font(): найден TTF / переход на системный шрифт / системный шрифт не поддерживает кириллицу	6, 7, 8
normalize_word(): нижний регистр и нормализация тире	9
load_words(): отсутствует файл / пустой файл / корректный utf-8-sig / недопустимый символ / ошибка декодирования и cp1251	10, 11, 12, 13, 14

Продолжение таблицы 3

Условие / ветвь	Тесты
load_record(): файл отсутствует / повреждён / корректный	15, 16, 17
save_record(): запись рекорда	18
display_word(): короткое слово / длинное без пропуска / длинное с пропуском / нет букв для пропуска	19, 20, 21, 22
load_img(): успешная загрузка / исключение загрузки / первое предупреждение / повтор без предупреждения	23, 24, 25, 26
button_images(): генерация трёх наборов вариантов изображения	27
init_audio(): успех / ошибка	28, 29
load_sound(): аудио не готово / файл не найден / успех / исключение загрузки звука	30, 31, 32, 33
play_sound(): звук отсутствует / успешное воспроизведение / воспроизведение с исключением	34, 35, 36
Button.draw(): обычная кнопка / наведение / активное изображение / нажатое состояние / вывод изображения	37, 38, 39, 40, 41
Button.click(): нажатие внутри / отпускание внутри / отпускание вне / событие игнорируется	42, 43, 44, 45
Word.update(): движение / отсутствие движения для угаданного слова	46, 47
Word.mark_guessed(): установка флага и таймера	48
Word.update_animation(): не угадано / таймер больше нуля / таймер завершён	49, 50, 51
Word.draw(): обычная отрисовка / анимация угаданного слова	52, 53
Game.load_music(): аудио не готово / файл не найден / успех / исключение загрузки	54, 55, 56, 57
ensure_music_loop(): запуск музыки / отсутствие запуска	58, 59

Продолжение таблицы 3

Условие / ветвь	Тесты
stop_music(): успех / исключение / музыка не загружена	60, 61, 62
make_buttons(): размер по умолчанию / размер по изображению	63, 64
set_diff(): лёгкая / средняя / сложная	65, 66, 67
start(), menu(), quit(), spawn(): основные переходы и сбросы	68, 69, 70, 71
submit(): найдено несколько совпадений / уже угадано / совпадений нет / рекорд не уменьшается	72, 73, 74, 75
update(): таймер предупреждения / отсутствие предупреждения / таймеры сердец / ранний выход	76, 77, 78, 79
update(): анимация активна / анимация завершена / слово в области / потеря жизни / последняя жизнь / жизни уже отсутствуют	80, 81, 82, 83, 84
event(): одновременные 2 события / закрытие окна / генерация слова в игре / генерация вне игры / кнопки меню / кнопки экрана окончания	85, 87, 88, 89, 90
event(): смешанный ввод / корректный ввод / только недопустимые символы	91, 92, 93
event(): Backspace / Enter / Escape / другая клавиша	94, 95, 96, 97
draw_text(): выравнивание по центру / без выравнивания по центру	98, 99
draw_input(): без предупреждения / с предупреждением	100, 101
heart_surface(): нет кадров / принудительно выключено / принудительно включено / активно / анимация / выключено / изображение отсутствует / масштабирование / без масштабирования	102, 103, 104, 105, 106, 107, 108, 109, 110

Окончание таблицы 3

Условие / ветвь	Тесты
draw_decor_note(): изображение есть / изображение отсутствует	111, 112
draw_hearts(): нет поверхностей / есть поверхности и задан y_center / y_center не задан	113, 114, 115
draw(): варианты главного фона, меню, игры, окончания игры, наличие и отсутствие монитора	116, 117, 118, 119, 120, 121, 122, 123
run(): основной цикл и завершение pygame после running=False	124
submit(): выбор слова функцией max() при равенстве координат Y у нескольких совпадений	125
event(): игнорирование текстового ввода (TEXTINPUT) в состояниях menu и game_over	126

2.11 Тестирование программы

Таблица 4 – Прохождение тестов

№	Фактический результат	Статус
1	Возвращается путь относительно каталога файла программы.	Пройден
2	Возвращается первый существующий путь.	Пройден
3	Возвращается None.	Пройден
4	Возвращается найденный путь к ресурсу.	Пройден
5	Возвращается None.	Пройден
6	Загружается пользовательский шрифт через pygame.font.Font.	Пройден
7	Возвращается системный шрифт.	Пройден
8	Печатается ошибка, pygame.quit(), sys.exit().	Пройден

Продолжение таблицы 4

№	Фактический результат	Статус
9	Возвращается «кошка-мышка».	Пройден
10	В консоли: «Ошибка: words.txt отсутствует или пуст.»; завершение.	Пройден
11	В консоли: «Ошибка: words.txt отсутствует или пуст.»; завершение.	Пройден
12	Возвращается список [«слово», «кошка-мышка»].	Пройден
13	Печатается ошибка о некорректных данных в строке 1; завершение.	Пройден
14	После ошибки кодировки выполняется повторное чтение cp1251; слова загружены.	Пройден
15	Возвращается 0.	Пройден
16	Возвращается 0.	Пройден
17	Возвращается 12.	Пройден
18	Файл содержит строку «7».	Пройден
19	Возвращается «дом», символ «_» не появляется.	Пройден
20	Возвращается «слово».	Пройден
21	Возвращается «с_ово».	Пройден
22	Возвращается «----».	Пройден
23	Изображение загружается и convert_alpha() выполняется.	Пройден
24	Исключение перехватывается, функция продолжает поиск; при отсутствии других вариантов возвращает None.	Пройден
25	Печатается предупреждение об отсутствии изображения, возвращается None.	Пройден
26	Предупреждение повторно не печатается, возвращается None.	Пройден
27	Возвращается кортеж из трёх изображений.	Пройден
28	_audio_ready = True.	Пройден

Продолжение таблицы 4

№	Фактический результат	Статус
29	Печатается предупреждение, <code>_audio_ready = False</code> .	Пройден
30	Возвращается <code>None</code> без попытки загрузки файла.	Пройден
31	Печатается предупреждение об отсутствии аудиофайла, возвращается <code>None</code> .	Пройден
32	Возвращается объект <code>Sound</code> .	Пройден
33	Печатается предупреждение о невозможности загрузить звук, возвращается <code>None</code> .	Пройден
34	Ничего не происходит, ошибки нет.	Пройден
35	Звук запускается.	Пройден
36	Исключение подавляется, программа продолжает работу.	Пройден
37	Рисуется серая кнопка и белый текст.	Пройден
38	Рисуется синяя кнопка.	Пройден
39	Отрисовывается <code>active_image</code> .	Пройден
40	Рисуется активное состояние цветом <code>YELLOW</code> .	Пройден
41	На экран выводится масштабированное <code>image</code> .	Пройден
42	<code>pressed</code> становится <code>True</code> , <code>action</code> пока не вызывается.	Пройден
43	Вызывается <code>action</code> , <code>pressed</code> становится <code>False</code> .	Пройден
44	<code>action</code> не вызывается, <code>pressed</code> становится <code>False</code> .	Пройден
45	Состояние не меняется, <code>action</code> не вызывается.	Пройден
46	Координата <code>y</code> увеличивается на 2.	Пройден
47	Координата <code>y</code> не меняется.	Пройден
48	<code>guessed=True</code> , <code>anim_timer=anim_duration</code> .	Пройден
49	Возвращается <code>False</code> .	Пройден
50	Возвращается <code>False</code> , слово остаётся на экране.	Пройден
51	Возвращается <code>True</code> , слово можно удалить.	Пройден
52	Рисуется обычное облачко со словом.	Пройден

Продолжение таблицы 4

№	Фактический результат	Статус
53	Рисуется исчезающее слово с alpha.	Пройден
54	Возвращается False.	Пройден
55	Печатается предупреждение, возвращается False.	Пройден
56	Возвращается True.	Пройден
57	Печатается предупреждение, возвращается False.	Пройден
58	Запускается <code>pygame.mixer.music.play(-1)</code> .	Пройден
59	Музыка не запускается.	Пройден
60	Музыка останавливается без ошибки.	Пройден
61	Исключение подавляется.	Пройден
62	Вызов <code>stop</code> не выполняется.	Пройден
63	Создаются кнопки с <code>default_size</code> и центрируются.	Пройден
64	Размер кнопки берётся из изображения.	Пройден
65	<code>diff_name="Лёгкая", diff=0.8</code> .	Пройден
66	<code>diff_name="Средняя", diff=1.5</code> .	Пройден
67	<code>diff_name="Сложная", diff=2.0</code> .	Пройден
68	<code>game_state="game", score=0, lives=5, timers=[0]*5, text="", words</code> очищен.	Пройден
69	<code>game_state="menu", text="", words</code> очищен, <code>record</code> сохраняется.	Пройден
70	<code>record</code> сохраняется, музыка останавливается, <code>running=False</code> .	Пройден
71	В <code>words</code> добавляется объект <code>Word</code> со скоростью 2.0.	Пройден
72	Помечается нижнее слово <code>y=50</code> , <code>score+1</code> , <code>record</code> обновляется, <code>text</code> очищается.	Пройден
73	<code>score</code> не меняется, <code>text</code> очищается.	Пройден
74	<code>score</code> не меняется, <code>text</code> очищается.	Пройден
75	<code>score=6</code> , <code>record</code> остаётся 10.	Пройден
76	Таймер предупреждения уменьшается до 2.	Пройден

Продолжение таблицы 4

№	Фактический результат	Статус
77	Таймер остаётся 0.	Пройден
78	Первый таймер уменьшается до 1.	Пройден
79	После таймеров функция завершает работу, слова не обновляются.	Пройден
80	Слово остаётся в списке alive.	Пройден
81	Слово удаляется из списка.	Пройден
82	Слово остаётся в active words.	Пройден
83	Слово удаляется, lives=4, запускается таймер сердца slot 4, game_over не наступает.	Пройден
84	lives становится 0, сохраняется record, играет end, game_state="game_over".	Пройден
85	События обрабатываются последовательно, ошибок нет.	Пройден
86	Вызывается quit(), running=False.	Пройден
87	Вызывается spawn().	Пройден
88	spawn() не вызывается, далее событие передаётся кнопкам меню.	Пройден
89	Вызывается click() соответствующей кнопки.	Пройден
90	Вызывается click() кнопки экрана окончания игры.	Пройден
91	В text добавляется «а-», input_warning_timer=FPS.	Пройден
92	В text добавляется «абв», предупреждение не включается.	Пройден
93	text не меняется, input_warning_timer=FPS.	Пройден
94	text становится «ко».	Пройден
95	Вызывается submit().	Пройден
96	Вызывается menu(), переход в меню.	Пройден
97	Состояние не меняется.	Пройден
98	Текст размещается по центру координат.	Пройден

Продолжение таблицы 4

№	Фактический результат	Статус
99	Текст размещается от левого верхнего угла.	Пройден
100	Рисуется поле ввода без предупреждения.	Пройден
101	Рисуется поле ввода и предупреждение.	Пройден
102	Возвращается None.	Пройден
103	Выбирается последний кадр HEART_FRAMES-1.	Пройден
104	Выбирается первый кадр.	Пройден
105	Выбирается первый кадр живого сердца.	Пройден
106	Выбирается промежуточный кадр анимации.	Пройден
107	Выбирается последний кадр погасшего сердца.	Пройден
108	Возвращается None.	Пройден
109	Возвращается уменьшенная поверхность.	Пройден
110	Возвращается поверхность без масштабирования.	Пройден
111	Изображение записки выводится на экран.	Пройден
112	Ничего не рисуется.	Пройден
113	Функция завершается без отрисовки.	Пройден
114	Рисуются существующие сердца, пропуски учитываются по spacing.	Пройден
115	baseline_y вычисляется как $50 + \text{ref_h} // 2$.	Пройден
116	Фон комнаты выводится на экран.	Пройден
117	Фон комнаты не выводится, ошибок нет.	Пройден
118	Рисуется экран меню с картинками заголовка и сложности.	Пройден
119	Рисуются текст «СЛОВОПАД» и текст «Сложность».	Пройден
120	Рисуется игровой экран, слова внутри clip области, счёт, рекорд, сердца и поле ввода.	Пройден
121	Рисуется экран окончания игры, рекорд, погасшие сердца и кнопки.	Пройден

Окончание таблицы 4

№	Фактический результат	Статус
122	Монитор выводится поверх фона.	Пройден
123	Монитор не выводится, ошибок нет.	Пройден
124	Выполняются <code>ensure_music_loop</code> , <code>tick</code> , обработка событий, <code>update</code> , <code>draw</code> ; после выхода <code>pygame.quit()</code> .	Пройден
125	Угадано одно слово, счет увеличен на 1.	Пройден
126	Текст в поле ввода (<code>self.text</code>) не изменяется, символы не добавляются.	Пройден

3 Заключение

В процессе прохождения практики выполнены следующие задачи:

- проведен сбор и анализ требований заказчика к программному средству;
- выполнена формализация выбранной предметной области;
- разработан проект программного средства;
- реализован проект программного средства;
- разработаны тесты и проведено тестирование программного средства;
- разработана и оформлена отчетная документация.

Таким образом, цели учебно-технологической (проектно-технологической) практики достигнуты.

В результате прохождения практики получены следующие профессиональные компетенции:

- способен разрабатывать языковые процессоры для формальных языков (ПК-7);
- способен создавать программные интерфейсы (ПК-8);
- способен использовать различные технологии разработки программного обеспечения (ПК-10).

4 Список использованных источников

1. Лутц, М. Изучаем Python : программирование игр, визуализация данных, веб-приложения / М. Луц. — 5-е изд. — Санкт-Петербург : Питер, 2020. — 832 с.
2. Свейгарт, Э. Придумываем и создаём игры на Python / Э. Свейгарт. — Москва : Эксмо, 2021. — 416 с.
3. Python Documentation [Электронный ресурс]. — Режим доступа: <https://docs.python.org/3/> (дата обращения: 10.06.2026).
4. Pygame Documentation [Электронный ресурс]. — Режим доступа: <https://www.pygame.org/docs/> (дата обращения: 10.06.2026).
5. Pygame Community Wiki [Электронный ресурс]. — Режим доступа: <https://www.pygame.org/wiki/> (дата обращения: 10.06.2026).